YOLO11 버스 객실 분석 파이프라인 Docker 환경 구축 보고서 (v1.2)

[방안 A] 독립 도커 컨테이너(yolo11-bus-distance:1.2) 환경 패키징 및 CPU 시연 · GPU 운영 가이드

프로젝트: miniproject4_containerized-ml-env-team1	작업 디렉터리: 13-yolo11-distance
구축 방식: [방안 A] 올인원 독립 실행형 컨테이너	기본 실행 환경: CPU 시연 모드 (경량 빌드)
적용 파이프라인: 이중 모델 융합 추론 + 데이터 증강/학습	이미지 버전: yolo11-bus-distance:1.2 (v1.2)

1. 도커화 아키텍처 전략: [방안 A] 독립 도커 이미지 환경 구성

본 프로젝트의 컴퓨터 비전 파이프라인(YOLO11 기반 객체 거리 측정 및 YOLOv8 Head 이중 모델 융합 추론, 데이터 증강 및 파인튜닝)을 어디서나 즉시 실행 및 검증할 수 있도록 [방안 A] 독립 도커 이미지 환경으로 구축하였습니다. 전체 웹 백엔드 서버에 결합되지 않고 독립 컨테이너로서 연구·검증·학습·추론 전체 사이클을 완전하게 재현합니다.

구분	[방안 A] 독립 도커 이미지 (채택)	[방안 B] 프로젝트 분리형 (기존)
컨테이너 구성	단독 yolo11-runner 올인원 서비스	trainer 및 backend 2개 서비스 분리
수행 범위	데이터 증강 → 분할 → 학습 → 이중 모델 추론 → 리포트	학습(trainer) / 웹 API 서빙(backend) 분할
장점	외부 서비스(DB/Nginx/Web) 종속 없이 즉시 시연 및 단독 재현 가능	대규모 마이크로서비스 운영 시 서비스별 스케일아웃에 유리
시연 적합성	최상 (명령어 1줄로 추론 및 시각화 결과 즉시 산출)	보통 (전체 Compose 스택 및 환경변수 사전 준비 필요)

2. Dockerfile 및 docker-compose.yml 상세 설계

(1) CPU 경량화 Dockerfile 최적화

• 기본 베이스로 python:3.11-slim을 채택하고, PyTorch 공식 CPU 휠(--index-url https://download.pytorch.org/whl/cpu)을 설치하여 CUDA 툴킷이 포함된 수 GB의 불필요한 용량과 빌드 시간을 획기적으로 절감하였습니다.

• requirements.txt 설치 시 grep -vE '^(torch|torchvision|nvidia-)==' 필터링을 거쳐 CPU 전용 라이브러리가 덮어쓰여지지 않도록 보호했습니다.

• 빌드 컨텍스트를 13-yolo11-distance 자체로 단순화하여 컨테이너 빌드 효율을 극대화했습니다.

(2) 호스트-컨테이너 간 볼륨 매핑 구조

호스트 경로	컨테이너 경로	권한	역할 및 데이터 내용
../backend/models	/app/backend/models	ro (읽기전용)	공유 가중치 (head_crowdhuman_n1280.pt, yolo11n.pt)
../backend/samples	/app/backend/samples	ro (읽기전용)	실제 버스 객실 검증 이미지 (bus_full.jpg, bus_empty.jpg)
./data	/app/data	rw (읽기/쓰기)	원천 이미지 및 증강 데이터 풀 (sample.jpg 등)
./datasets	/app/datasets	rw (읽기/쓰기)	YOLO 학습용 train/val 분할 데이터셋 (data.yaml)
./results	/app/results	rw (읽기/쓰기)	추론 결과 이미지 및 PDF/HTML 리포트 산출물

3. 파이프라인 코드 유연화 (컨테이너·호스트 완전 호환)

도커 환경과 기존 로컬 개발 환경 모두에서 무수정 동작할 수 있도록 파이프라인 코드를 유연화하였습니다.

• 환경 변수 우선 탐색: CV_MODELS_DIR 환경 변수를 지원하여 컨테이너 마운트 경로(/app/backend/models)를 우선 참조하고, 미지정 시 로컬 상대 경로(../../backend/models)로 자동 폴백합니다.

• 실행 디바이스 자동 감지: DualModelVacancyPipeline(..., device=None) 초기화 시 torch.cuda.is_available()을 자동 판별하여 GPU 가용 환경에서는 'cuda', CPU 환경에서는 'cpu'로 설정하며, 추론(infer) 시 predict(..., device=self.device)에 직접 전달되도록 개선했습니다.

4. CPU 모드 시연 실행 절차 (Command Reference)

시연은 docker compose run 명령어를 통해 즉시 수행할 수 있습니다:

```
1) 이미지 빌드:
docker compose -f 13-yolo11-distance/docker-compose.yml build

2) 기본 샘플 이미지(data/sample.jpg) 이중 모델 융합 추론:
docker compose -f 13-yolo11-distance/docker-compose.yml run --rm yolo11-runner ₩
  python scripts/test_dual_vacancy.py --source /app/data/sample.jpg --out /app/results/output_sample.jpg

3) 백엔드 버스 실사(bus_full.jpg) 융합 추론:
docker compose -f 13-yolo11-distance/docker-compose.yml run --rm yolo11-runner ₩
  python scripts/test_dual_vacancy.py --source /app/backend/samples/bus_full.jpg --out /app/results/dual_full.jpg
```

5. ★ GPU(CUDA) 환경 전환 핵심 수정 가이드 (Checklist)

차후 GPU 가속 서버 및 클라우드 환경으로 이전할 때 수정해야 하는 4대 포인트를 코드 레벨에서 완벽히 기록하였습니다.

항목	CPU 시연 모드 (현재)	GPU 환경 전환 시 수정 내용
[수정 1] Dockerfile 베이스 및 Torch	FROM python:3.11-slim RUN pip install --index-url https://.../cpu torch torchvision	교체: FROM pytorch/pytorch:2.2.0-cuda12.1-cudnn8-runtime ※ 공식 이미지에 CUDA/Torch가 내장되므로 CPU 휠 설치 RUN 라인 주석 처리
[수정 2] docker-compose deploy 리소스	deploy 설정 주석 처리 image: yolo11-bus-distance:cpu	주석 해제 및 태그 변경: image: yolo11-bus-distance:gpu deploy.resources.reservations.devices: - driver: nvidia, count: all, capabilities: [gpu]
[수정 3] 단독 docker run CLI 플래그	docker run --rm ...	--gpus all 플래그 추가: docker run --gpus all --rm ...
[수정 4] 스크립트 실행 디바이스 설정	추론: 자동 감지(CPU 할당) 학습: python scripts/train_bus.py --device cpu	추론: torch.cuda.is_available() 자동 감지(GPU 자동 할당) 학습: python scripts/train_bus.py --device 0 (명시 할당)

6. 구축 결과 및 산출물 요약

- 산출물 파일:
  1) 13-yolo11-distance/Dockerfile (CPU 경량 빌드 + GPU 주석 가이드)
  2) 13-yolo11-distance/docker-compose.yml (호스트-컨테이너 간 볼륨 매핑 서비스)
  3) 13-yolo11-distance/DOCKER_GUIDE.md (시연 및 GPU 전환 상세 매뉴얼)
  4) 13-yolo11-distance/src/dual_pipeline.py (환경 변수 및 디바이스 자동 감지 개선)
- 기대 효과: 로컬 시연 시 수 초 이내의 가벼운 컨테이너 구동이 가능하며, 호스트 OS 환경에 구애받지 않고 일관된 버스 혼잡도 이중 모델 추론 결과를 재현할 수 있습니다.